
Onder de motorkap: een introductie tot de CH552-microcontroller

Een rustige inleiding in computerarchitectuur voor ICT-studenten

Versie 0.5 · Datum: 14-7-2026 · Auteur: E. vd Oetelaar (edwin@oetelaar.com)

Disclaimer — aansprakelijkheid

De auteur aanvaardt géén enkele aansprakelijkheid, in welke vorm dan ook, voor het gebruik van dit document of voor eventuele fouten die het bevat. Alle informatie in dit document dient door de gebruiker zelf te worden gecontroleerd. Dit is een work in progress en kan fouten bevatten. Gebruik is volledig op eigen risico.

Over dit document

Je hebt waarschijnlijk al eens een ledje laten knipperen op een Arduino, of een sensorwaarde uitgelezen in Python. Dat is een prima start — maar in beide gevallen werd er *heel veel* machinerie voor je verborgen. Je schreef `digitalWrite(13, HIGH)` en iets, ergens, zette een echte spanning op een pin. Wat was dat "iets"? Wat voert jouw code eigenlijk uit?

Dit document opent de motorkap. We gebruiken als voorbeeldmachine een kleine, goedkope en verfrissend eenvoudige chip — de **CH552**. Hij is modern genoeg om nuttig te zijn (hij spreekt zelfs USB), maar zijn brein is gebaseerd op de **8051**, een ontwerp uit 1980 dat zó eenvoudig is dat je het hele ding in je hoofd kunt houden. Dat maakt hem perfect om van te leren.

Aan het eind zou je in concrete termen moeten begrijpen:

- wat een microcontroller eigenlijk *is*, en hoe hij verschilt van de computer op je bureau;
- wat er gebeurt tussen de C-code die je schrijft en de chip die iets doet;
- wat een **compiler**, **assembler** en **linker** zijn;
- de kernonderdelen van een CPU: de **program counter**, de **accumulator**, de **registers**, de **stack** en de **stack pointer**, het **statusregister** en zijn **flags** (carry, zero en aanverwanten);
- hoe de CPU beslissingen neemt met **conditionele sprongen** (conditional jumps).

Je hoeft de CH552-datasheet niet uit je hoofd te leren. Het doel is begrip, geen naslagwerk.

Inhoudsopgave

1. **Over dit document**
 2. **Deel 1 — Wat is een microcontroller?** — een complete computer op één chip; kennismaking met de CH552
 3. **Deel 2 — Van jouw code naar de chip** — machinecode, assembler, compiler, linker, flashen
 4. **Deel 3 — In de CPU: het programmeermodel** — fetch-decode-execute, program counter, accumulator, registers, geheugen
 5. **Deel 4 — Het statusregister en zijn flags** — carry, zero, overflow, pariteit, bank-selectie
 6. **Deel 5 — Beslissingen nemen: conditionele sprongen** — jumps, van C naar sprongen, lussen
 7. **Deel 6 — De stack en de stack pointer** — LIFO, SP, `CALL / RET`, interrupts
 8. **Deel 7 — Alles bij elkaar** — een compleet voorbeeldprogramma
 9. **Woordenlijst**
 10. **Waar je hierna heen kunt**
-

Deel 1 — Wat is een microcontroller?

Een complete computer op één chip

De machine waarop je dit leest, is een *general-purpose computer* (computer voor algemeen gebruik). Hij heeft een krachtige CPU, gigabytes aan RAM op aparte chips, een schijf, een scherm, een besturingssysteem (Windows, macOS, Linux), en draait vele programma's tegelijk.

Een **microcontroller** (vaak afgekort tot MCU) neemt de *essentiële* ingrediënten van een computer — een processor, geheugen en een manier om met de buitenwereld te praten — en propt die op één *enkele* chip. Standaard is er geen besturingssysteem, geen harde schijf en meestal geen scherm. Hij draait precies één programma, dat jij erop zet, keer op keer, eeuwig, totdat de stroom eraf gaat.

Een microcontroller bevat doorgaans:

- een **CPU** (het deel dat instructies uitvoert);
- **programmeergeheugen** (niet-vluchtig flashgeheugen dat je programma bewaart, ook zonder stroom);
- **datageheugen** (vluchtig RAM dat variabelen vasthoudt tijdens het draaien);

- **randapparatuur (peripherals)** — ingebouwde hardwareblokken om met de wereld te communiceren: GPIO-pinnen, timers, een USB-controller, een analoog-naar-digitaal-omzetter (ADC), seriële poorten, enzovoort.

Het hele idee is *embedded control* (ingebbedde besturing): een microcontroller zit in een product — een magnetron, een toetsenbord, speelgoed, een thermostaat — en bestuurt dat stilletjes.

Maak kennis met de CH552

De **CH552** is een 8-bits microcontroller van het Chinese bedrijf WCH. Hij is populair bij hobbyisten en in goedkope producten omdat hij ruim onder een euro kost, een **USB**-interface bevat, en met gratis gereedschap geprogrammeerd kan worden.

Belangrijke feiten over de CH552 (de exacte getallen verschillen licht binnen de CH551/CH552-familie):

Kenmerk	CH552
CPU-kern	"E8051" — een verbeterde, 8051-compatibele kern
Databreedte	8-bits
Kloksnelheid	tot 24 MHz
Programmageheugen (flash)	~16 KB
Intern RAM	256 bytes
On-chip xRAM	1 KB
Randapparatuur	USB 2.0, GPIO, timers, PWM, ADC, UART, SPI, I ² C

"8-bits" betekent dat de CPU van nature werkt met getallen van 8 bits breed — één **byte**, waarden 0-255. Als je grotere getallen nodig hebt, verwerkt de CPU die in meerdere stappen, byte voor byte. Vergelijk dat met de 64-bits processor in je laptop, die 64 bits tegelijk verwerkt.

Het woord "**verbeterd**" (**enhanced**) is belangrijk. De oorspronkelijke 8051 uit 1980 had 12 klokcycli nodig om een typische instructie uit te voeren. De E8051-kern van de CH552 voert ongeveer 79% van de instructies in één enkele klokcyclus uit, waardoor hij bij dezelfde kloksnelheid ruwweg **8 tot 15 keer sneller** is dan een klassieke 8051. De *architectuur* — de registers, instructies en het programmeermodel dat we zo gaan leren — is hetzelfde. Alleen de snelheid is veranderd. Precies daarom is de 8051 goed om te leren: een 45 jaar oud ontwerp zit nog steeds in nieuwe chips.

Deel 2 — Van jouw code naar de chip

Voordat we in de CPU kijken, beantwoorden we de vraag die Arduino en Python netjes voor je verborgen: **hoe wordt mijn code iets dat de chip uitvoert?**

De CPU begrijpt alleen getallen

Een CPU kan geen C, Python of Nederlands lezen. Diep vanbinnen voert hij **machinecode** uit: een lange reeks bytes die in het programmeergeheugen staat. Elke instructie is maar één of enkele bytes. Op de 8051 betekent de byte `0x04` bijvoorbeeld "tel één op bij de accumulator" (de instructie `INC A`). De CPU haalt die byte op, herkent hem, doet de ophoging, en gaat door naar de volgende byte.

Programma's rechtstreeks als bytes schrijven zou ellendig zijn. Daarom hebben we lagen gereedschap die mensvriendelijke tekst omzetten in die bytes.

Assembleertaal en de assembler

De dunst mogelijke laag boven ruwe bytes is **assembleertaal (assembly)**. Die geeft elke machine-instructie een korte, leesbare naam (een *mnemonic*). In plaats van `0x04` schrijf je `INC A`. Assembly is een bijna één-op-één, voor mensen leesbare vorm van machinecode.

Het gereedschap dat assembly-tekst omzet in machinecode-bytes heet een **assembler**. Assembly → machinecode. Dat is (bijna) alles wat het doet.

```
; Een piepklein stukje 8051-assembly
MOV  A, #5      ; zet het getal 5 in de accumulator
INC  A          ; A is nu 6
ADD  A, #10     ; A is nu 16
```

Elk van die regels komt direct overeen met een handvol bytes in het geheugen. De taak van de assembler is in wezen een opzoektabel: zet elke mnemonic om in de bijbehorende byte(s).

Hogere programmeertalen en de compiler

Assembly is leesbaar, maar nog steeds pijnlijk laag-niveau — je beheert elk register en elk geheugenadres met de hand. Daarom schrijven we meestal in een **hogere programmeertaal** zoals C. Met C kun je `x = x + 10;` schrijven zonder je druk te maken over welk register `x` bevat.

Een **compiler** is het gereedschap dat een hogere taal vertaalt naar een lagere — meestal naar assembly (die daarna geassembleerd wordt) of rechtstreeks naar machinecode. Voor de CH552 is de standaard gratis compiler **SDCC** (de Small Device C Compiler), die de 8051-familie als doel kan nemen.

De keten tot nu toe is dus:

```
C-broncode —(compiler)→ assembly —(assembler)→ machinecode
```

Een compiler doet veel meer dan een simpele opzoektabel. Hij moet beslissen welke variabelen in welke registers wonen, `if`-statements en lussen omzetten in sprongen, en optimaliseren. Eén regel C kan een dozijn machine-instructies worden.

De linker

Echte programma's zijn verdeeld over meerdere bestanden en gebruiken kant-en-klare bouwstenen (bibliotheken) — bijvoorbeeld code die weet hoe je met de USB-peripheral praat. Elk bestand wordt apart gecompileerd tot een *objectbestand* (een brok machinecode met enkele lege plekken, want bestand A weet nog niet waar de functies van bestand B in het geheugen zijn beland).

De **linker** is het gereedschap dat alle objectbestanden en bibliotheken aan elkaar naait tot één definitief programma, waarbij die lege plekken worden ingevuld — het bepaalt het uiteindelijke geheugenadres van elke functie en variabele zodat de onderdelen elkaar kunnen aanroepen. De uitvoer is één image, klaar om in het flashgeheugen van de chip te worden geplaatst.

```
bestand1.c -(compileren)→ bestand1.rel  ↙
bestand2.c -(compileren)→ bestand2.rel  └-(linker)→ eindprogramma (.hex / .bin)
bibliotheek ───────────→ lib.rel        ↗
```

Flashen

Dat eindprogramma staat nog op je pc. De laatste stap, **flashen** (of *programmeren*), kopieert het naar het flashgeheugen van de microcontroller via USB. Vanaf dat moment draait de CH552 *jouw* programma telkens als hij wordt ingeschakeld. De CH552 heeft handig een ingebouwde USB-bootloader, dus er is geen speciale programmeerhardware nodig.

De volledige pijplijn

```
mijn_programma.c
  | compiler (SDCC)      ← vertaalt C naar een lager niveau
  ▼
assembly / objectcode
  | assembler           ← zet mnemonics om in machinecode-bytes
  ▼
objectbestanden (.rel)
  | linker              ← combineert bestanden + bibliotheken, kent adressen toe
  ▼
eind-image (.hex)
  | flasher (via USB)    ← kopieert de image naar de chip
  ▼
CH552 draait jouw programma
```

Toen je op "Upload" klikte in de Arduino IDE, gebeurde dit *allemaal* op de achtergrond. Nu weet je wat die lagen waren.

Deel 3 — In de CPU: het programmeermodel

Nu openen we de CPU zelf. Een CPU is in de kern een heel snelle, heel gehoorzame en heel domme machine die eeuwig één lus herhaalt:

***fetch** de volgende instructie → **decode** wat hij betekent → **execute** hem → herhaal.*

Dit heet de **fetch-decode-execute-cyclus**, en hij stopt nooit zolang de chip stroom heeft.

Om zijn werk te doen heeft de CPU een handvol piepkleine, zeer snelle opslagplekken die in het silicium zijn ingebouwd, **registers** genoemd. Registers zijn *geen* RAM — er zijn er maar een paar, ze hebben vaste taken, en het is de snelste opslag die de CPU heeft. De verzameling registers en hun gedrag heet het **programmeermodel** van de CPU. Laten we dat van de 8051 bekijken, dat is de CH552.

De program counter (PC)

De **program counter** is een register dat het geheugenadres bevat van de *volgende* instructie die opgehaald moet worden. Zie het als de vinger van de CPU die naar een regel in het programma wijst.

De fetch-decode-execute-lus werkt zo: de CPU leest de instructie waar de PC naar wijst, en *verhoogt de PC automatisch* naar de daaropvolgende instructie. Standaard marcheert de CPU dus recht door je programma heen, instructie na instructie, in volgorde.

De PC is wat een **sprong (jump)** mogelijk maakt: als we de waarde in de PC met opzet *veranderen*, gaat de CPU vanaf een andere plek verder. Dat ene idee — de PC overschrijven — is de basis van elke lus, elke `if` en elke functie-aanroep in elk programma dat ooit geschreven is. We komen er in Deel 5 op terug.

Op de 8051 is de PC 16 bits breed, en daarom kan de chip tot 64 KB programmeergeheugen adresseren ($2^{16} = 65\,536$ adressen).

De accumulator (A)

De **accumulator**, geschreven als **A** (of **ACC**), is het belangrijkste register van de 8051. Het is het "kladblok" van de CPU — de standaardplek waar rekenkunde en logica gebeuren.

Op veel oudere CPU's, waaronder de 8051, *stromen* de meeste berekeningen *dóór* de accumulator: je laadt een waarde in A, doet er iets mee, en het resultaat verschijnt in A. Om twee getallen op te tellen zet je er bijvoorbeeld één in A en tel je het andere daar *bij op*; de som belandt weer in A.

```
MOV  A, #20      ; A = 20
ADD  A, #22      ; A = A + 22 → A = 42
```

De naam is historisch: hij "accumuleert" tussenresultaten. Een moderne CPU heeft veel gelijkwaardige general-purpose registers, maar de 8051 leunt sterk op dit ene speciale register — wat het eigenlijk *makkelijker* maakt om te leren: er is één duidelijke plek waar de actie gebeurt.

Er is ook een hulpregister **B**, vooral gebruikt samen met A voor vermenigvuldigen en delen.

De general-purpose registers R0-R7

Naast A en B geeft de 8051 je acht general-purpose registers, **R0 tot en met R7**, om werkwaarden in te bewaren. Slim genoeg zijn dit geen aparte hardware — het zijn simpelweg de eerste bytes van het interne RAM, aangeboden onder handige namen.

De 8051 heeft zelfs **vier banken** van R0-R7, en je kunt kiezen welke bank "actief" is met twee bits in het statusregister (hieronder uitgelegd). Dit is een handige truc om in één keer een hele set werkvariabelen te wisselen — bijvoorbeeld wanneer er een interrupt optreedt, zodat de interruptroutine de registers van het hoofdprogramma niet overschrijft.

```
MOV  R0, #100    ; bewaar 100 in register R0
MOV  A, R0       ; kopieer R0 naar de accumulator
```

Geheugen: een korte plattegrond

Het helpt om te weten waar dingen wonen. De 8051 houdt programma en data in *aparte* ruimtes (dit heet een **Harvard-architectuur**, in tegenstelling tot het **von Neumann**-ontwerp van je pc, waar code en data één geheugen delen):

- **Programmageheugen (flash):** je instructies. Alleen-lezen tijdens het draaien. ~16 KB op de CH552.
- **Intern RAM:** 256 bytes snel on-chip datageheugen. De onderste 128 bytes bevatten de registerbanken, een bit-adresseerbaar gebied en de stack; de bovenste 128 bytes overlappen qua adres met de special-function registers (hieronder), maar worden op een andere manier benaderd.
- **Special Function Registers (SFR's):** een blok registers op adressen 80h-FFh dat het *bedieningspaneel* van de chip is. Naar een SFR schrijven doet iets fysieks — een pin hoog zetten, een timer starten, het USB-blok instellen. A, B, de stack pointer en het statusregister wonen hier allemaal. Op de CH552 heeft WCH extra SFR's toegevoegd om zijn eigen peripherals (zoals USB) te besturen, bovenop de standaard 8051-set.
- **xRAM:** 1 KB extra on-chip RAM voor grotere data.

De belangrijkste conclusie voor nu: **registers zijn weinig en snel; RAM is groter en iets trager; flash bevat het programma.**

Deel 4 — Het statusregister en zijn flags

Wanneer de CPU een berekening doet, gaat het *resultaat* naar de accumulator — maar de CPU noteert ook enkele *feiten over* dat resultaat in een speciaal register. Op de 8051 is dat het **Program Status Word (PSW)**, het **statusregister** van de chip.

Het PSW is één byte, en elk bit is een aparte **flag** — een ja/nee-sigitaal over de laatste bewerking of over de huidige modus van de CPU. Deze flags zijn hoe de CPU dingen "onthoudt" zoals *is die optelling overgelopen?* en, cruciaal, ze zijn waar conditionele sprongen naar kijken om beslissingen te nemen.

Hier is het 8051-PSW, bit voor bit:

Bit	Naam	Betekenis
PSW.7	CY	Carry-flag
PSW.6	AC	Auxiliary carry (overdracht uit de lage nibble; gebruikt voor BCD-rekenen)
PSW.5	F0	General-purpose gebruikersflag (vrij voor jou te gebruiken)
PSW.4	RS1	Registerbank-selectiebit 1
PSW.3	RS0	Registerbank-selectiebit 0
PSW.2	OV	Overflow-flag
PSW.1	—	Vrij te definiëren gebruikersflag
PSW.0	P	Pariteit van de accumulator (1 als A een oneven aantal 1-bits heeft)

Laten we naar de belangrijkste voor beginners kijken.

De carry-flag (CY)

De **carry-flag** vangt het "extra" bit op dat er bovenaan afvalt wanneer een 8-bits berekening niet in 8 bits past.

Onthoud dat A één byte bevat: waarden 0–255. Wat gebeurt er als je $200 + 100$ optelt? Het echte antwoord is 300, wat niet in 8 bits past. De accumulator houdt de onderste 8 bits ($300 - 256 = 44$) en de **carry-flag wordt op 1 gezet** om te melden "er was overloop voorbij 255; er droeg een 1 uit de top over."

```
MOV  A, #200
ADD  A, #100    ; echt resultaat 300; A = 44, CY = 1 (carry eruit!)

MOV  A, #10
ADD  A, #20     ; resultaat 30 past prima; A = 30, CY = 0
```

Dit is precies zoals het "onthouden van de 1" wanneer je $7 + 5 = 12$ met de hand optelt: je schrijft 2 op en houdt 1 over. De carry-flag is hoe de CPU byte-grote optellingen aan elkaar rijgt om grotere getallen aan te kunnen, én hoe hij unsigned overloop meldt.

De zero-flag — een bijzonder geval dat het begrijpen waard is

Hier is een subtiliteit die mensen verrast, en die een prachtig leermoment vormt. Op veel CPU's is er een aparte **zero-flag** die automatisch wordt gezet zodra een resultaat gelijk aan nul uitkomt. De klassieke 8051 **heeft geen zero-flag-bit in zijn PSW**.

In plaats daarvan test de 8051 de accumulator *rechtstreeks* op nul, op het moment van de sprong zelf, met de instructies `JZ` ("spring als A nul is") en `JNZ` ("spring als A niet nul is"). Het concept "is het resultaat nul?" bestaat absoluut — het wordt alleen op aanvraag geëvalueerd in plaats van in een flag-bit opgeslagen.

Dus als we het over "de zero-flag" hebben als algemeen architectuurbegrip, houd dan in gedachten dat het een *concept* is dat verschillende CPU's verschillend invullen. Op de 8051 wordt de vraag "is het nul?" beantwoord doordat de `JZ / JNZ`-instructies rechtstreeks naar de accumulator kijken. Dit is een goede herinnering dat "computerarchitectuur" een familie van verwante ontwerpen is, geen vast regelboek.

De overflow-flag (OV) en pariteit (P)

De **overflow-flag (OV)** is de *signed* neef van de carry-flag. Wanneer je bytes als getallen met teken behandelt (−128 tot +127), wordt OV gezet als het resultaat niet in dat signed bereik past. Carry is voor unsigned overflow; overflow is voor signed. Beginners kunnen dit gerust opzijleggen tot ze signed rekenkunde in detail bestuderen.

De **pariteit-flag (P)** meldt simpelweg of de accumulator op dit moment een oneven of even aantal 1-bits bevat. Hij werd historisch gebruikt voor goedkope foutcontrole in communicatie. Je zult hem zelden rechtstreeks aanraken.

De bank-selectiebits (RS1, RS0)

Weet je nog, de vier banken van R0–R7? **RS1 en RS0** zijn de twee PSW-bits die kiezen welke bank actief is. `00` kiest bank 0, `01` bank 1, enzovoort. Deze bits horen bij de "status" omdat ze deel uitmaken van de huidige *modus* van de CPU, niet van een feit over een berekening.

Deel 5 — Beslissingen nemen: conditionele sprongen

We hebben nu alle onderdelen om uit te leggen hoe een CPU *beslissingen* neemt — hoe een domme rechttoe-rechtaan-uitvoerder `if`-statements, lussen en vertakkingen produceert.

Onvoorwaardelijke sprongen

Herinner je uit Deel 3 dat de program counter normaal zichzelf ophoogt, zodat instructies in volgorde draaien. Een **sprong (jump)**-instructie overschrijft dat: hij laadt

een nieuw adres in de PC, zodat de uitvoering ergens anders verdergaat. Een **onvoorwaardelijke sprong** springt altijd:

```
SJMP ergens ; spring altijd naar het label 'ergens'
```

Dit alleen al geeft je een oneindige lus — het hart van bijna elk microcontrollerprogramma, dat doorgaans eeuwig draait:

```
lus: ; ... doe iets ...  
    SJMP lus ; ga terug naar de top, voor altijd
```

Conditionele sprongen

Een **conditionele sprong** springt *alleen als een bepaalde voorwaarde waar is*, en valt anders door naar de volgende instructie. De voorwaarde is meestal een **flag** in het statusregister, of een rechtstreekse test van de accumulator. Dit is het enige mechanisme achter elke `if`, `while` en `for` in je C-code.

De veelgebruikte 8051 conditionele sprongen:

Instructie	Springt als...
JZ	de accumulator nul is
JNZ	de accumulator niet nul is
JC	de carry -flag 1 is
JNC	de carry-flag 0 is
JB	een opgegeven bit 1 is
JNB	een opgegeven bit 0 is
CJNE	twee waarden niet gelijk zijn (en het zet ook de carry om te tonen welke groter was)
DJNZ	na het verlagen van een register, dit nog niet nul is

Van C naar sprongen: een uitgewerkt voorbeeld

Bekijk deze gewone C-code:

```
if (x == 0) {  
    y = 1;  
}
```

De CPU heeft geen `if`-instructie. De compiler zet dit om in een *test* gevolgd door een *conditionele sprong*. In 8051-assembly zou het er zo uit kunnen zien:

```

MOV  A, x          ; laad x in de accumulator
JNZ  skip          ; als A NIET nul is, spring voorbij de body
MOV  y, #1         ; (draait alleen wanneer x == 0)
skip: ; ... programma gaat hier verder ...

```

Lees het zorgvuldig. Om de body uit te voeren "wanneer x gelijk is aan nul," springt de compiler *weg* wanneer x *niet* nul is. Die omkering is volkomen normaal in machinecode en is precies het soort boekhouding dat een compiler voor je doet.

Lussen met een teller

DJNZ ("decrement and jump if not zero" — verlaag en spring als niet nul) bouwt een tellende lus in één enkele instructie — een favoriet op de 8051:

```

MOV  R2, #10       ; herhaal 10 keer
opnieuw: ; ... body van de lus ...
DJNZ R2, opnieuw   ; R2 = R2 - 1; als R2 ≠ 0, spring terug naar 'opnieuw'
; valt hierheen door na 10 iteraties

```

Dit is het assembly-skelet van een `for (i = 10; i != 0; i--)`-lus. Elke lus die je ooit hebt geschreven, wordt teruggebracht tot een conditionele sprong als deze.

Deel 6 — De stack en de stack pointer

Het laatste kernbegrip is de **stack** — het mechanisme dat het ene stukje code in staat stelt een ander aan te roepen en veilig terug te keren.

Welk probleem lost de stack op?

Stel dat je `main`-code een functie `leesSensor()` aanroept. Wanneer `leesSensor` klaar is, moet de CPU terugkeren naar *precies* de instructie na de aanroep in `main`. Maar de CPU heeft maar één program counter. Hoe onthoudt hij waar hij naar terug moet? En wat als `leesSensor` zelf weer een andere functie aanroept? We moeten een *ketting* van terugkeeradressen onthouden, meest-recent-eerst.

Het antwoord is een **stack**: een geheugengebied dat wordt gebruikt in **last-in, first-out (LIFO)**-volgorde, precies als een stapel borden. Je **pusht** iets bovenop; je **popt** de bovenste eraf. Het laatste dat je pushte, is het eerste dat je popt.

De stack pointer (SP)

Ja — de 8051, en dus de CH552, heeft een **stack pointer**. Het is een 8-bits SFR genaamd **SP** die het intern-RAM-adres van de huidige top van de stack bevat.

- **Push:** de CPU verhoogt SP en schrijft dan een byte op dat adres. (Op de 8051 groeit de stack *omhoog*, richting hogere adressen.)

- **Pop:** de CPU leest de byte op SP en verlaagt dan SP.

Bij reset begint SP op adres 07h, dus de stack begint net boven de registerbanken in het interne RAM.

CALL en RET : de stack in actie

Wanneer de CPU een CALL-instructie uitvoert (een functie-aanroep), doet hij twee dingen: hij **pusht de huidige PC** (het terugkeeradres) op de stack, en laadt dan de PC met het adres van de functie. Wanneer de functie klaar is met RET, **popt de CPU het terugkeeradres** terug in de PC — en de uitvoering gaat verder precies waar hij was gebleven.

```
CALL leesSensor    ; push terugkeeradres, spring in leesSensor
MOV  A, R7         ; ← uitvoering keert HIER naartoe terug

; ... elders ...
leesSensor:
    ; ... doe het werk, laat een resultaat in R7 ...
    RET            ; pop terugkeeradres, spring terug naar aanroeper
```

Omdat de stack LIFO is, werkt dit zelfs wanneer functies functies aanroepen die functies aanroepen: elke CALL pusht een terugkeeradres, elke RET popt het meest recente, en de ketting rolt perfect in omgekeerde volgorde af.

Je kunt de stack ook zelf gebruiken om *tijdelijk* een waarde op te slaan met PUSH en die later terug te halen met POP — handig als je een register even voor iets anders nodig hebt maar niet mag verliezen wat erin zat:

```
PUSH ACC           ; bewaar de accumulator op de stack
; ... gebruik A vrij voor iets anders ...
POP  ACC           ; herstel de oorspronkelijke waarde van de accumulator
```

Dezelfde stack-machinerie is ook wat **interrupts** laat werken: wanneer een peripheral dringend aandacht nodig heeft, pusht de CPU automatisch de PC, draait een interruptroutine, en popt de PC om verder te gaan — het programma merkt niet eens dat het onderbroken werd.

Deel 7 — Alles bij elkaar

Laten we elk begrip verbinden met één klein, compleet idee: een programma dat telt hoeveel bytes in een kleine lijst nul zijn. Het oefent de accumulator, een register, de flags, een conditionele sprong en een lus.

```

MOV R0, #lijst      ; R0 wijst naar het begin van de data
MOV R2, #8          ; R2 = hoeveel items te controleren (de lusteller)
MOV R7, #0          ; R7 = onze lopende telling van nullen (begint op 0)

volgende:
MOV A, @R0          ; laad de byte waar R0 naar wijst in de accumulator
JNZ nietnul         ; als A NIET nul is, sla het ophogen over
INC R7              ; A was nul → tel hem mee

nietnul:
INC R0              ; schuif de pointer naar de volgende byte
DJNZ R2, volgende   ; R2 = R2 - 1; als nog niet nul, herhaal de lus
; wanneer we hier doorvallen, bevat R7 het aantal nullen

```

Volg het in je hoofd met wat je nu weet:

- **R0** loopt door het geheugen; **R2** telt de lus af; **R7** verzamelt het antwoord.
- `MOV A, @R0` haalt een byte in de **accumulator** — het kladblok waar de test gebeurt.
- `JNZ` is een **conditionele sprong** die vraagt "is A nul?" en beslist of `INC R7` wordt uitgevoerd.
- `DJNZ` combineert een verlaging en een **conditionele sprong** om de lus te bouwen, waarbij de **program counter** wordt gewijzigd om terug te gaan naar `volgende` tot de teller nul bereikt.
- Achter de schermen werken elke `MOV` en `INC` de toestand van de CPU bij, en het geheel draait binnen de eeuwige fetch-decode-execute-cyclus.

Dat is een echte computer, die echt werk doet, met niets verborgen. Alles wat je op hogere niveaus gebruikt — Python-lijsten, Arduino-lussen, hele besturingssystemen — is uiteindelijk opgebouwd uit precies deze bewegingen: laad een register, test een flag, spring op een voorwaarde, push en pop de stack.

Woordenlijst

Accumulator (A/ACC) — het belangrijkste kladblok-register van de CPU, waar de meeste rekenkundige en logische resultaten verschijnen.

Assembler — een gereedschap dat assembly-mnemonics omzet in machinecode-bytes.

Assembleertaal (assembly) — een voor mensen leesbare, bijna één-op-één tekstvorm van machinecode.

Carry-flag (CY) — een statusbit dat wordt gezet wanneer een unsigned berekening buiten het bereik van de byte overloopt.

Compiler — een gereedschap dat een hogere taal (zoals C) vertaalt naar een lagere (assembly of machinecode).

Conditionele sprong — een instructie die de program counter alleen wijzigt als een voorwaarde (een flag, of A gelijk aan nul) waar is.

Flag — één enkel bit in het statusregister dat een feit over de CPU of de laatste bewerking vastlegt.

Flashgeheugen — niet-vluchtig programmeergeheugen dat zijn inhoud behoudt zonder stroom.

Linker — een gereedschap dat apart gecompileerde objectbestanden en bibliotheken combineert tot één eindprogramma, en definitieve geheugenadressen toekent.

Machinecode — de ruwe bytes die de CPU daadwerkelijk uitvoert.

Microcontroller (MCU) — een complete kleine computer (CPU + geheugen + peripherals) op één chip, ontworpen om één embedded programma te draaien.

Overflow-flag (OV) — als de carry-flag, maar voor *signed* rekenkunde.

Program counter (PC) — het register dat het adres bevat van de volgende uit te voeren instructie.

Register — een piepkleine, zeer snelle opslagplek die in de CPU is ingebouwd.

Stack — een last-in-first-out geheugengebied, vooral gebruikt om terugkeeradressen van functie-aanroepen op te slaan.

Stack pointer (SP) — het register dat het adres van de top van de stack bevat.

Statusregister (PSW) — het register dat de flags van de CPU bevat.

Zero-flag — het concept "was het resultaat nul?"; let op dat de klassieke 8051 geen zero-flag-bit heeft en in plaats daarvan de accumulator rechtstreeks test met `JZ / JNZ`.

Waar je hierna heen kunt

Deze inleiding geeft je het mentale model; de echte kennis komt door het zelf te doen. Een paar concrete vervolgstappen:

- **Probeer de toolchain.** Installeer **SDCC** (de Small Device C Compiler) en een flash-hulpprogramma zoals `chprog` of de Arduino-IDE met CH55x-ondersteuning. Compileer een eenvoudig blink-programma en flash het via USB naar een CH552-board. Zie bewust de pijplijn `.c` → compileren → linken → `.hex` → flashen in actie; het is precies wat we in Deel 2 beschreven.
- **Lees wat assembly.** Vraag SDCC om de gegenereerde assembly voor een kleine C-functie (bijvoorbeeld met de optie om een `.asm`-uitvoer te bewaren). Koppel elke regel C aan de sprongen, registerbewegingen en flags die je nu herkent. Dit is de snelste manier om te zien wat een compiler écht voor je doet.
- **Speel met de flags.** Schrijf een klein programma dat twee getallen optelt en daarna, afhankelijk van de carry-flag, een led aan- of uitzet. Zo maak je het abstracte begrip "flag" tastbaar.
- **Verken de peripherals.** De begrippen uit dit document (registers, flags, lezen en schrijven) breiden rechtstreeks uit naar de special function registers (SFR's) die

GPIO, timers, PWM en USB besturen. Hardware besturen op een microcontroller is, uiteindelijk, niets anders dan de juiste speciale registers lezen en schrijven.

- **Bouw iets kleins.** Laat een led knipperen, lees een knop uit, of stuur een tekst over USB naar je pc. Elk klein project maakt de theorie concreet en bouwt vertrouwen op.

Neem de tijd, wees niet bang om dingen stuk te maken (een microcontroller van een halve euro is vergevingsgezind), en raadpleeg altijd de officiële datasheet als de details ertoe doen.

Bronnen en verder lezen

- WCH, *CH552/CH551 8-bit Enhanced USB MCU Datasheet* — <https://akizukidenshi.com/goodsaffix/CH552.pdf>
- WCH, *CH552/CH551 Manual* — https://cdn.hackaday.io/files/1696717259204064/CH552%20Datasheet_C111367.zh-CN.en.pdf
- *Introduction to the CH552G Microcontroller* — https://altbier.us/ch552g/Intro_to_the_CH552G_Microcontroller.pdf
- CNX Software, *Cocket Nova CH552 development board* — <https://www.cnx-software.com/2024/10/17/6-cocket-nova-ch552-development-board-features-ch552g-8-bit-mcu-with-an-enhanced-8051-core/>
- SDCC C-voorbeelden voor de CH552 — https://github.com/Cesarbautista10/CH55x_SDCC_Examples